DRCS Engine — Deployment Runbooks

Step-by-step procedures for deploying, monitoring, and rolling back the DRCS engine in production.

Runbook 1 — Initial Production Deployment

Prerequisite: All 107 tests passing, TypeScript build clean, staging verification complete.

Step 1 — Provision Production Database (PostgreSQL)

```
# Create PostgreSQL database
createdb drcs_production

# Generate connection string
# Format: postgresql://user:password@host:5432/drcs_production
export DATABASE_URL="postgresql://drcs_user:SECURE_PASSWORD@prod-db.example.com:5432/
drcs_production"
```

Verify:

```
psql $DATABASE_URL -c "SELECT version();"
# Should return PostgreSQL version
```

Step 2 — Update Prisma Schema for PostgreSQL

Edit `prisma/schema.prisma`:

```
datasource db {
-  provider = "sqlite"
+  provider = "postgresql"
  url      = env("DATABASE_URL")
}
```

Why: SQLite is dev-only. PostgreSQL is required for production multi-tenant workloads.

Step 3 — Deploy Migrations

```
# Generate Prisma client
npx prisma generate

# Apply all migrations to production DB
npx prisma migrate deploy
```

Verify:

```
npx prisma migrate status
# Should show: "Database schema is up to date"
```

If migrations fail:

1. Check `DATABASE_URL` is correct
2. Verify database user has CREATE/ALTER permissions
3. Review migration files in `prisma/migrations/`
4. Manually apply failed migration if needed

Step 4 — Seed Production Tenant Data

CRITICAL: Do NOT seed demo data in production. Use production-ready tenant configs.

```
# Run production seed (replace with your tenant's real data)
npm run seed:zilly
```

Post-seed verification:

```
npx prisma studio
# Open in browser, verify:
# - CategorySchema has entries for your tenant
# - AssetRegistry has assets with real file_path values
# - TenantConfig exists
```

Step 5 — Set Production Environment Variables

Create `.env.production`:

```
# Database
DATABASE_URL="postgresql://drcs_user:SECURE_PASSWORD@prod-db.example.com:5432/
drcs_production"

# LLM API
ABACUS_API_KEY="prod-api-key-here"

# Server
PORT=3000
NODE_ENV="production"

# Default tenant (optional, can be overridden per request)
DRCS_TENANT="your-production-tenant"

# Logging (optional)
LOG_LEVEL="info"
```

Verify:

```
source .env.production
echo $DATABASE_URL
echo $ABACUS_API_KEY
# Both should print values
```

Step 6 — Build for Production

```
npm run build
# Outputs to dist/
```

Verify:

```
ls -lh dist/
# Should see:
# - orchestrator/index.js
# - gates/*/index.js
# - llm/index.js
# - persistence/index.js
# - etc.
```

Step 7 — Start Production Server

```
# Option A: Direct (foreground, for testing)
npm run serve

# Option B: PM2 (background, production-recommended)
pm2 start dist/server/index.js --name drcs-engine
pm2 save
pm2 startup  # Configure PM2 to restart on system reboot
```

Verify:

```
curl http://localhost:3000
# Should return the web console HTML
```

Step 8 — Run Production Health Check

```
# Test natural language prompt
npm run evaluate -- --prompt "test production deployment"

# Expected output:
# DECISION: PUBLISH (or BLOCKED, depending on governance config)
# FILE: <file_path or null>
# CAPTION: <caption or generated text>
```

If health check fails:
1. Check `ABACUS_API_KEY` is valid
2. Verify tenant data is seeded
3. Check logs: `pm2 logs drcs-engine`
4. Test LLM connectivity: `curl -H "Authorization: Bearer $ABACUS_API_KEY" https://routellm.abacus.ai/v1/chat/completions`

Step 9 — Configure Reverse Proxy (Optional)

If exposing to the internet, use nginx or similar:

```
server {
    listen 80;
    server_name drcs.example.com;

    location / {
        proxy_pass http://localhost:3000;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
    }
}
```

Verify:

```
curl http://drcs.example.com
# Should return the web console
```

Step 10 — Set Up Monitoring (See Runbook 3)

- [ ] Configure log aggregation (e.g., CloudWatch, Datadog, Sentry)
- [ ] Set up uptime monitoring (e.g., Pingdom, UptimeRobot)
- [ ] Create dashboards for key metrics (C5 errors, C3 blocks, latency)
- [ ] Set up alerts (LLM failures, database errors, high latency)

Runbook 2 — Rollback Procedure

When: Production deployment causes errors, performance degradation, or data issues.

Step 1 — Identify Last Known Good Version

```
git log --oneline
# Find the commit hash of the last stable release
# Example: abc1234 "fix: map prompt to category name"
```

Step 2 — Stop Current Production Server

```
pm2 stop drcs-engine
```

Step 3 — Revert Code to Last Known Good

```
git checkout abc1234
npm install  # Restore previous dependencies
npm run build
```

Step 4 — Rollback Database Migrations (If Needed)

CAUTION: Only rollback if the new migrations broke production.

```
# List migrations
npx prisma migrate status

# Revert the last migration
npx prisma migrate resolve --rolled-back 20260805140000_add_file_path
```

Verify:

```
npx prisma migrate status
# Should show rolled-back migration
```

Step 5 — Restart Production Server

```
pm2 restart drcs-engine
```

Step 6 — Run Health Check

```
npm run evaluate -- --prompt "rollback health check"
```

Verify: Expected output matches pre-rollback behavior.

Step 7 — Notify Stakeholders

- Post-mortem: What broke? Why?
- Root cause: Bad migration? LLM API change? Logic bug?
- Fix timeline: When will the issue be resolved?

Runbook 3 — Production Monitoring Setup

Goal: Detect issues before users report them.

Metrics to Track

Metric	Tool	Alert Threshold
LLM Errors (C5 ESCALATE_FAILED)	Application logs	> 5% of requests
C3 Repetition Blocks	Application logs	> 20% of requests (may need new content)
C6 Governance REJECT rate	Application logs	> 10% of requests (policy may be too strict)
Average Latency (evaluatePrompt)	APM (e.g., New Relic, Datadog)	> 5 seconds (LLM timeout?)
Database Query Time	PostgreSQL slow query log	> 1 second
Uptime	Pingdom / UptimeRobot	< 99.9%

Log Aggregation Setup (Example: CloudWatch)

Install CloudWatch agent:

```
wget https://s3.amazonaws.com/amazoncloudwatch-agent/ubuntu/amd64/latest/amazon-cloudwatch-agent.deb
dpkg -i amazon-cloudwatch-agent.deb
```

Configure log shipping:

```
{
  "logs": {
    "logs_collected": {
      "files": {
        "collect_list": [
          {
            "file_path": "/var/log/pm2/drcs-engine-out.log",
            "log_group_name": "/drcs/production",
            "log_stream_name": "{instance_id}"
          }
        ]
      }
    }
  }
}
```

Start agent:

```
amazon-cloudwatch-agent-ctl -a fetch-config -m ec2 -c file:/opt/aws/amazon-cloudwatch-agent/etc/config.json -s
```

Alert Setup (Example: Datadog)

Install Datadog agent:

```
DD_API_KEY=<your-key> bash -c "$(curl -L https://s3.amazonaws.com/dd-agent/scripts/install_script.sh)"
```

Create alert:

1. Go to Datadog → Monitors → New Monitor
2. Select “Log Monitor”
3. Query: `source:drcs-engine status:error`
4. Alert condition: `> 5 errors in 5 minutes`
5. Notification: Slack / email / PagerDuty

Custom Metrics (Optional)

Instrument the orchestrator to emit metrics:

```
import { Counter, Histogram } from 'prom-client';

const c5ErrorCounter = new Counter({
  name: 'drcs_c5_escalate_failed_total',
  help: 'Total C5 ESCALATE_FAILED outcomes'
});

const verdictLatency = new Histogram({
  name: 'drcs_verdict_latency_seconds',
  help: 'evaluatePrompt latency distribution'
});

// In orchestrator:
if (c5Result.action === 'ESCALATE_FAILED') {
  c5ErrorCounter.inc();
}

const end = verdictLatency.startTimer();
const verdict = await evaluate(request, tenant_id);
end();
```

Expose metrics endpoint:

```
// src/server/index.ts
import { register } from 'prom-client';

if (req.url === '/metrics') {
  res.setHeader('Content-Type', register.contentType);
  res.end(await register.metrics());
  return;
}
```

Scrape with Prometheus:

```
scrape_configs:
  - job_name: 'drcs-engine'
    static_configs:
      - targets: ['localhost:3000']
```

Runbook 4 — Adding a New Tenant in Production

When: Onboarding a new customer/deployment.

Step 1 — Prepare Tenant Data

Create a seed file: `src/seeds/<tenant-name>.ts`

```ts
import { prisma } from '../persistence/client';

export async function seedNewTenant() {
  const tenant_id = 'new-tenant';

  // 1. Tenant config
  await prisma.tenantConfig.upsert({
    where: { tenant_id },
    update: {},
    create: {
      tenant_id,
      max_repetition_count: 3,
      rolling_window_days: 30,
    },
  });

  // 2. Category schema
  await prisma.categorySchema.upsert({
    where: { tenant_id_category_id: { tenant_id, category_id: 'example-cat' } },
    update: {},
    create: {
      tenant_id,
      category_id: 'example-cat',
      name: 'Example Category',
      asset_list: ['asset_001'],
      protected_flag: false,
      prestocked_flag: true,
    },
  });

  // 3. Assets
  await prisma.assetRegistry.upsert({
    where: { tenant_id_asset_id: { tenant_id, asset_id: 'asset_001' } },
    update: {},
    create: {
      tenant_id,
      asset_id: 'asset_001',
      tag: 'canonical',
      category: 'example-cat',
      caption: 'Example caption',
      file_path: 'media/new-tenant/example.png',
      content_hash: 'sha256:...',
      sealed_hash: 'sha256:...',
    },
  });

  console.log(`✅ Seeded tenant: ${tenant_id}`);
}
```

Step 2 — Run Seed in Production

```bash
# Add seed command to package.json
# "seed:new-tenant": "ts-node src/seeds/new-tenant.ts"

npm run seed:new-tenant
```

Verify:

```
npx prisma studio
# Check that new tenant's data appears in all 3 tables
```

Step 3 — Test New Tenant

```
npm run evaluate -- --tenant new-tenant --prompt "test new tenant"
```

Expected: Verdict uses new tenant's categories/assets.

Step 4 — Document Tenant-Specific Config

Create `docs/tenants/<tenant-name>.md`:

```
# Tenant: new-tenant

## Categories
- example-cat: [Description]

## Assets
- asset_001: Example content

## Deployment Instances
- Production: deployed 2026-08-05

## Contact
- Owner: [Name]
- Email: [Email]
```

Runbook 5 — Handling LLM Outages

When: Abacus AI routeLLM is unreachable or returning errors.

Step 1 — Detect the Outage

Symptom: High rate of `C5 ESCALATE_FAILED` outcomes in logs.

Check LLM health:

```
curl -H "Authorization: Bearer $ABACUS_API_KEY" \
  https://routellm.abacus.ai/v1/chat/completions \
  -d '{"model":"gpt-4o-mini","messages":[{"role":"user","content":"health check"}]}'
```

If returns HTTP 5xx or timeout: LLM is down.

Step 2 — Enable Fallback Mode (If Implemented)

If you've built a fallback (e.g., cache recent captions, or use a static caption list):

```
# Set env var to enable fallback
export DRCS_LLM_FALLBACK="true"
pm2 restart drcs-engine
```

If no fallback exists: C5 will fail, but the engine won't crash. Users will get `outcome: ESCALATE_FAILED` instead of `GENERATED`.

Step 3 — Notify Users (If Customer-Facing)

Message:

```
We're experiencing issues with our content generation service.
Existing content is still available, but new content generation may be delayed.
We're monitoring the situation and will update you when service is restored.
```

Step 4 — Monitor LLM Recovery

Set up a cron job or monitoring script:

```
# Check every 5 minutes
*/5 * * * * curl -s -H "Authorization: Bearer $ABACUS_API_KEY" \
  https://routellm.abacus.ai/v1/chat/completions \
  -d '{"model":"gpt-4o-mini","messages":[{"role":"user","content":"test"}]}' \
  && echo "LLM is UP" || echo "LLM is DOWN"
```

Step 5 — Disable Fallback Mode Once LLM Recovers

```
unset DRCS_LLM_FALLBACK
pm2 restart drcs-engine
```

Runbook 6 — Database Backup & Restore

When: Routine backups or disaster recovery.

Daily Backup (Automated)

PostgreSQL dump:

```
# Cron job (runs daily at 2 AM)
0 2 * * * pg_dump $DATABASE_URL | gzip > /backups/drcs_$(date +\%Y\%m\%d).sql.gz
```

Verify backups exist:

```
ls -lh /backups/
# Should see drcs_20260805.sql.gz, drcs_20260806.sql.gz, etc.
```

Restore from Backup

Restore entire database:

```
gunzip < /backups/drcs_20260805.sql.gz | psql $DATABASE_URL
```

Restore specific table:

```
gunzip < /backups/drcs_20260805.sql.gz | psql $DATABASE_URL -c "COPY AssetRegistry
FROM STDIN"
```

Test Restore (Dry Run)

Create test database:

```
createdb drcs_restore_test
export TEST_DATABASE_URL="postgresql://user:pass@host:5432/drcs_restore_test"
```

Restore to test DB:

```
gunzip < /backups/drcs_20260805.sql.gz | psql $TEST_DATABASE_URL
```

Verify:

```
psql $TEST_DATABASE_URL -c "SELECT COUNT(*) FROM AssetRegistry;"
# Should match production count
```

Runbook 7 — Performance Tuning

When: Latency increases above acceptable thresholds (> 5 seconds per request).

Identify Bottleneck

Check logs for slow operations:

```
pm2 logs drcs-engine | grep "took [0-9]\{4,\} ms"
# Look for operations taking > 1000ms
```

Common bottlenecks:
- LLM call timeout (C5)
- Slow database query (C2 category resolution, C3 usage log query)
- Large asset registry scan (C4 resolution)

Optimize LLM Calls

Reduce timeout:

```
// src/llm/index.ts
const timeout = options.timeout_ms ?? 10000; // Reduce from 20000 to 10000
```

Cache recent LLM responses:

```
const promptCache = new Map<string, string>();

export async function callLLM(prompt: string, options: CallLlmOptions = {}) {
  if (promptCache.has(prompt)) {
    return promptCache.get(prompt)!;
  }
  const result = await /* ... actual LLM call ... */;
  promptCache.set(prompt, result);
  return result;
}
```

Optimize Database Queries

Add indexes to frequently queried columns:

```
-- C3 usage log queries
CREATE INDEX idx_usage_log_asset_deployed_at ON UsageLog(asset_id, deployed_at);

-- C2 category name lookups
CREATE INDEX idx_category_schema_name ON CategorySchema(name);
```

Verify index usage:

```
EXPLAIN ANALYZE SELECT * FROM UsageLog WHERE asset_id = 'test' AND deployed_at >
'2026-01-01';
-- Should show "Index Scan" not "Seq Scan"
```

Scale Horizontally

Deploy multiple instances behind a load balancer:

```
# Instance 1
PORT=3001 pm2 start dist/server/index.js --name drcs-engine-1

# Instance 2
PORT=3002 pm2 start dist/server/index.js --name drcs-engine-2

# Load balancer (nginx)
upstream drcs_backend {
    server localhost:3001;
    server localhost:3002;
}
```

Common Issues & Solutions

Issue	Symptom	Solution
LLM 401 Unauthorized	C5 always fails	Check `ABACUS_API_KEY` is valid
C2 "No category matches"	All prompts blocked	Seed category data for tenant
C3 blocks everything	`failed_closed: true`	Check usage log integrity; clear corrupt records
`file_path` is null	Verdict returns no media	Seed `file_path` in `AssetRegistry`
Prisma client not found	Build fails	Run `npx prisma generate`
Migrations fail	DB schema out of sync	Manually apply migrations; check permissions

---

Remember: These runbooks are living documents. Update them every time you encounter a new production issue or find a better solution.